

bok25

基
礎
班

、考研数据結構



图的遍历

从图中某一顶点出发，按照某种方法对图中所有顶点进行访问且仅访问一次



图的遍历深度优先遍历 (DFS)

(Depth First Search)

```
// 声明一个布尔数组visited，用于标记图中的顶点是否已被访问，大小为MVNum，即图中顶点的最大数量。
bool visited[MVNum];

// DFS函数，用于从指定的顶点v开始，深度优先遍历图G。
void DFS(Graph G, int v)
{
    // 打印当前访问的顶点编号v。
    cout<<v;

    // 将当前顶点v标记为已访问。
    visited[v]=true;

    // 循环遍历v的所有邻接顶点。
    for(w=FirstAdjVex(G,v);w>=0;w=NextAdjVex(G, v, w))
    {
        // 如果邻接顶点w未被访问，则递归调用DFS函数，以w为起点继续进行深度优先遍历。
        if(!visited[w]) DFS(G,w);
    }
}
```



图的遍历深度优先遍历 (DFS)

(Depth First Search)

```
// 声明一个布尔数组visited，用于标记图中的顶点是否已被访问，大小为MVNum，即图中顶点的最大数量。  
bool visited[MVNum];
```

```
// DFS函数，用于从指定的顶点v开始，深度优先遍历图G。
```

```
void DFS(Graph G, int v)
```

```
{
```

```
    // 打印当前访问的顶点编号v。
```

```
    cout<<v;
```

```
    // 将当前顶点v标记为已访问
```

```
    visited[v]=true;
```

```
    // 循环遍历v的所有邻接顶点。
```

```
    for(w=FirstAdjVex(G,v);w>=0;w=NextAdjVex(G, v, w))
```

```
{
```

```
        // 如果邻接顶点w未被访问，则递归调用DFS函数，以w为起点继续进行深度优先遍历。
```

```
        if(!visited[w]) DFS(G,w);
```

```
    }
```

```
}
```

FirstAdjVex(G, v)：返回图G中顶点v的第一个邻接点的索引或标识符。在这段代码中， $w \geq 0$ 表示存在邻接点

NextAdjVex(G, v, w)：返回图G中与顶点v的邻接点w之后的下一个邻接点

图的遍历深度优先遍历 (DFS) 代码流程

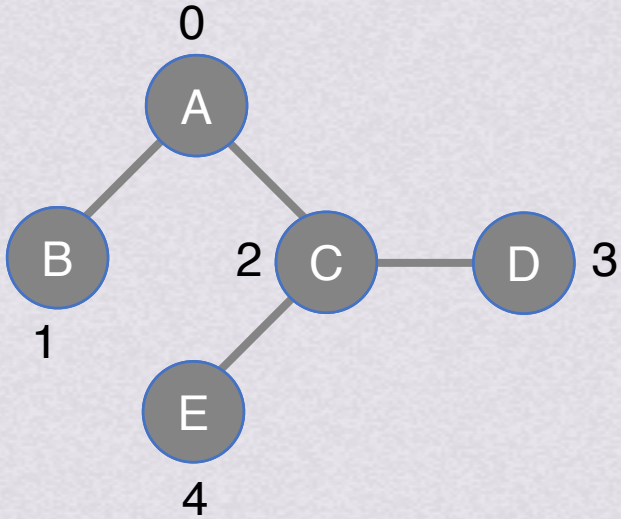
(Depth First Search)

```
// 声明一个布尔数组visited，用于标记图中的顶点是否已被访问，大小为MVNum，即图中顶点的最大数量。
bool visited[MVNum]; 声明辅助数组visited[5]

// DFS函数，用于从指定的顶点v开始，深度优先遍历图G。
void DFS(Graph G, int v)
{
    // 打印当前访问的顶点编号v。
    cout<<v;

    // 将当前顶点v标记为已访问。
    visited[v]=true;

    // 循环遍历v的所有邻接顶点。
    for(w=FirstAdjVex(G, v); w<=0; w=NextAdjVex(G, v, w))
    {
        // 如果邻接顶点w未被访问，则递归调用DFS函数，以w为起点继续进行深度优先遍历。
        if(!visited[w]) DFS(G, w);
    }
}
```



递归栈

true	true	true	true	true
------	------	------	------	------

数组下标 0 1 2 3 4

深度优先遍历输出序列： 0 1 2 3 4

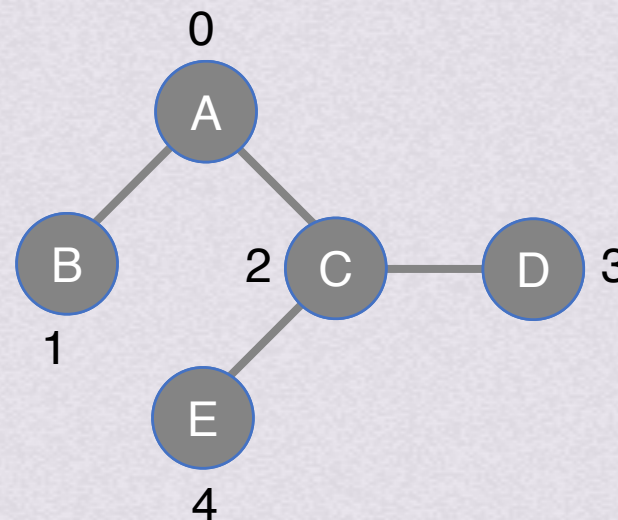


图的遍历深度优先遍历 (DFS) 代码流程

(Depth First Search)

深度优先遍历输出序列：0 1 2 3 4

*注：深度优先遍历输出序列不唯一，若得到
02341；02431；01243也是正确答案



	true	true	true	true	true
数组下标	0	1	2	3	4



图的遍历深度优先遍历 (DFS) 代码流程

(Depth First Search)

// 声明一个布尔数组visited, 用于标记图中的顶点是否已被访问, 大小为MVNum, 即图中顶点的最大数量。

```
bool visited[MVNum];
```

// DFS函数, 用于从指定的顶点v开始, 深度优先遍历图G。

```
void DFS(Graph G, int v)
```

```
{
```

```
    // 打印当前访问的顶点编号v。
```

```
    cout<<v;
```

```
    // 将当前顶点v标记为已访问。
```

```
    visited[v]=true;
```

```
    // 循环遍历v的所有邻接顶点。
```

```
    for(w=FirstAdjVex(G, v); w>=0; w=NextAdjVex(G, v, w))
```

```
    {
```

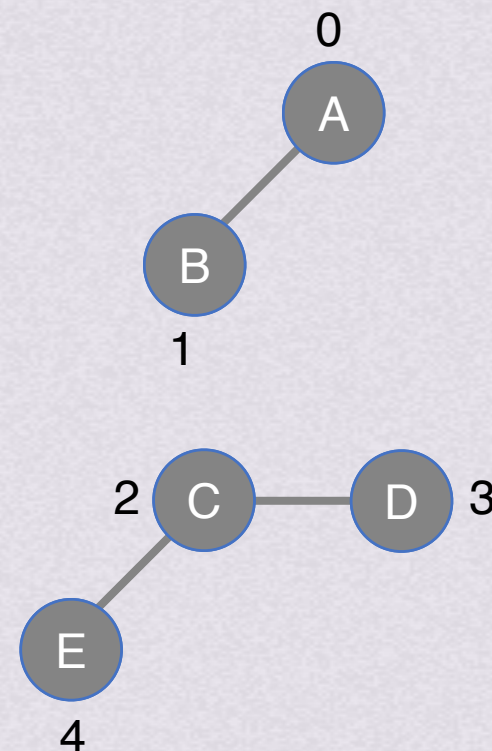
```
        // 如果邻接顶点w未被访问, 则递归调用DFS函数, 以w为起点继续进行深度优先遍历。
```

```
        if(!visited[w]) DFS(G, w);
```

```
    }
```

```
}
```

```
void DFSTraverse(Graph G)
{
    for(v=0; v<G.vexnum; ++v)
        visited[v]=false;
    for(v=0; v<G.vexnum; ++v)
        if(!visited[v]) DFS(G, v);
}
```



图H

对于右边图H, 执行DFSTraverse会调用两次DFS函数

每调用一次DFS将遍历一个连通分量, 有多少次遍历, 就说明图中有多少个连通分量



图的遍历

深度优先遍历 (DFS)

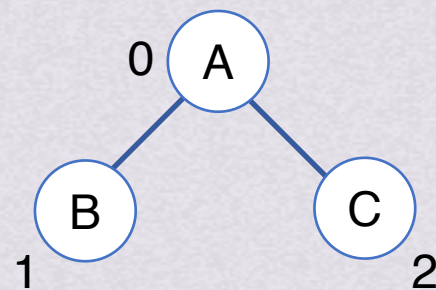
(Depth First Search)

图的DFS类似树的先序遍历



图的遍历深度优先遍历 (DFS) 代码流程-2

(Depth First Search)

图G₅

DFS输出序列: 0 1 2

	true	true	true
数组下标	0	1	2

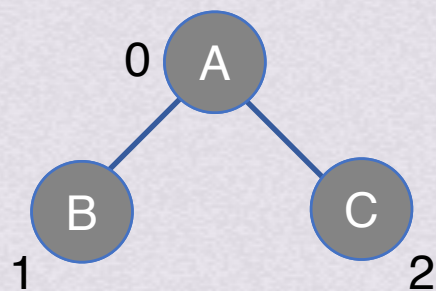
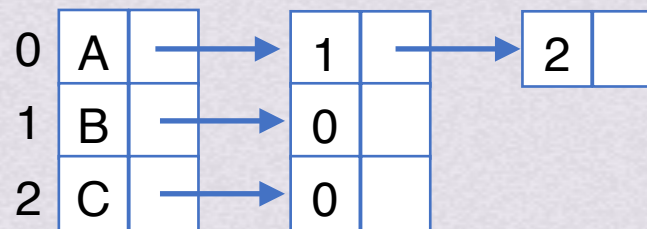
0	1	1
1	0	0
1	0	0

对应邻接矩阵G



图的遍历深度优先遍历 (DFS) 代码流程-2

(Depth First Search)

图G₅

对应邻接表G

DFS输出序列: 0 1 2



图的遍历深度优先遍历 (DFS)

(Depth First Search)

遍历耗费的时间取决于图采用的存储结构

邻接矩阵 $O(n^2)$

邻接表 $O(n+e)$



图的遍历广度优先遍历 (BFS)

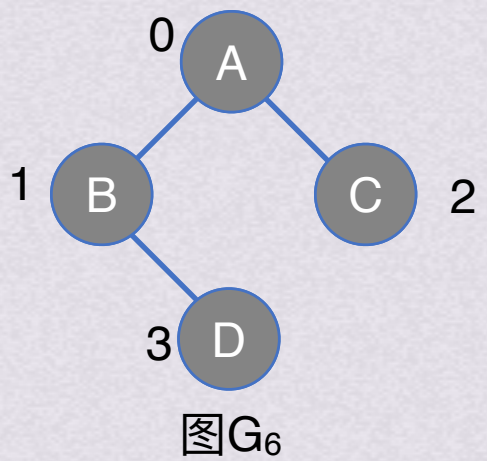
(Breadth First Search)

图的广度优先搜索遍历类似于树的层次遍历

图的遍历广度优先遍历 (BFS)

```
bool visited[MVNum]; //辅助数组 (Breadth First Search)
// 使用邻接矩阵表示的图的广度优先搜索算法
void BFS_AM(AMGraph G, int v) 传入G,0
{
    InitQueue(Q); // 定义队列 创建辅助队列
    // 输出当前访问的顶点编号v
    cout << v; 输出0
    // 标记顶点v为已访问
    visited[v] = true; visited[0]=true
    // 顶点v入队
    EnQueue(Q,v); 顶点0入队
    while(!QueueEmpty(Q))
    {
        // 队首顶点出队
        DeQueue(Q,u);
        // 遍历图G中的所有顶点，以检查任何未访问的邻接顶点
        for(int w = 0; w < G.vexnum; w++)
        {
            // 如果顶点u到顶点w有边 (G.arcs[u][w] != 0表示u和w之间有连接) 且顶点w未被访问
            if((G.arcs[u][w] != 0) && (!visited[w]))
            {
                // 输出当前访问的顶点编号w
                cout << w;
                // 标记顶点w为已访问
                visited[w] = true;
                // 顶点w入队
                EnQueue(Q,w);
            }
        }
    }
}
```

数组下标	0	1	2	3
	true	true	true	true



0	1	1	0
1	0	0	1
1	0	0	0
0	1	0	0

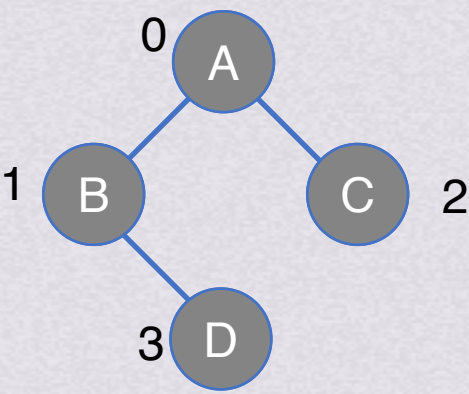
对应邻接矩阵G

← 辅助队列

BFS遍历序列: 0 1 2 3

图的遍历广度优先遍历 (BFS)

(Breadth First Search)



图G₆

BFS遍历序列: 0 1 2 3

0	1	1	0
1	0	0	1
1	0	0	0
0	1	0	0

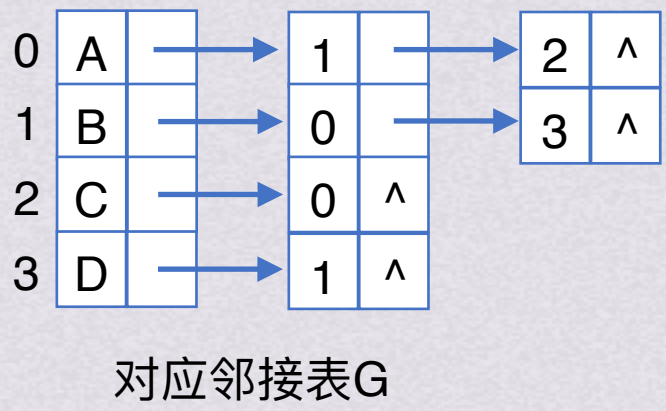
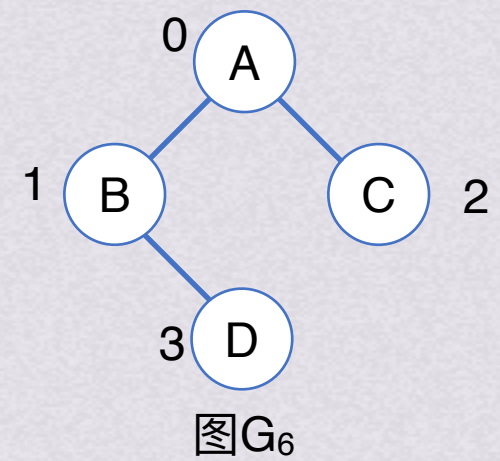
对应邻接矩阵G

true	true	true	true
------	------	------	------

数组下标 0 1 2 3

图的遍历广度优先遍历 (BFS)

```
bool visited[MVNum]; //辅助数组
// 使用邻接表表示的图的广度优先搜索算法
void BFS_AL(ALGraph G, int v) {
    InitQueue(Q); // 定义队列
    // 输出当前访问的顶点编号v
    cout << v;
    // 标记顶点v为已访问
    visited[v] = true;
    // 将顶点v加入队列
    EnQueue(Q, v);
    // 当队列不为空时循环
    while (!QueueEmpty(Q)) {
        // 从队列中取出一个顶点
        DeQueue(Q, u);
        // 获取顶点u的邻接表的头指针
        p = G.vertices[u].firstarc;
        // 循环遍历顶点u的所有邻接点
        while (p != NULL) {
            // 获取当前遍历到的邻接顶点的编号
            w = p->adjvex;
            // 如果该邻接顶点未被访问, 则进行操作
            if (!visited[w]) {
                // 输出访问的顶点编号w
                cout << w;
                // 标记顶点w为已访问
                visited[w] = true;
                // 将顶点w加入队列
                EnQueue(Q, w);
            }
            // 移动到顶点u的下一个邻接点
            p = p->nextarc;
        }
    }
}
```





图的遍历广度优先遍历 (BFS)

(Breadth First Search)

BFS遍历序列: 0 1 2 3

时间复杂度: 邻接矩阵 $O(n^2)$

邻接表 $O(n+e)$

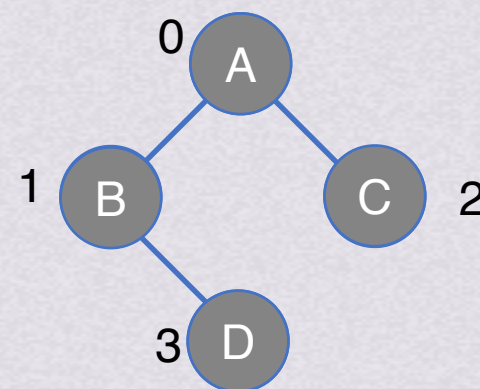
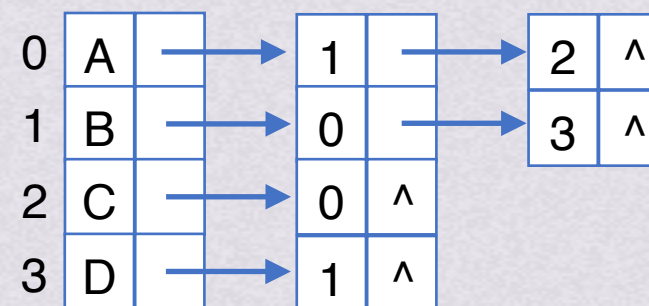


图 G_6



对应邻接表G